Greedy Algorithms — Complete Guide

Enhanced Beginner Edition — original content preserved, with added plain-English explanations, brute-force reasoning, and "why we optimize" insight for every problem.

228. Assign Cookies

Understanding the Problem NEW

You have children each with a 'greed factor' (the minimum cookie size that would satisfy them) and cookies of various sizes. Assign cookies to children to maximize how many children end up satisfied (each cookie can go to at most one child).

Brute Force Approach NEW

Try every possible way of assigning cookies to children and count satisfied children for each assignment — combinatorially expensive, exploring assignments that are obviously wasteful (like giving a huge cookie to an easily-satisfied child, wasting its size on someone who didn't need it).

Why We Optimize NEW

Sort both the children's greed factors and the cookie sizes. Then greedily walk through both sorted lists together: give the smallest available cookie that's big enough to satisfy the current least-greedy child — if it's big enough, both move forward; if not, only the cookie pointer moves forward (that cookie is too small for anyone remaining, or rather, that child needs a bigger cookie found later). Matching the smallest sufficient cookie to the least-greedy unsatisfied child (rather than 'wasting' a big cookie on someone who needed little) leaves the largest cookies available for the more demanding children — this greedy 'smallest fit' choice, after sorting, achieves the optimal number of satisfied children in O(n log n) instead of exploring every assignment combination.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Assign cookies to children to maximize satisfied children.

Visual Diagram

```
Children Greed: [1, 2, 3]
          Cookies Size: [1, 1]
          Output: 1 (only child with greed 1 gets cookie)

          Sort both:
          Children: [1, 2, 3]
```

```
Cookies: [1, 1]

i=0 (greed=1), j=0 (cookie=1): 1>=1 → satisfied, i=1, j=1
i=1 (greed=2), j=1 (cookie=1): 2>1 → no, j=2
i=1, j=2: end
Result: 1

Children Greed: [1, 2], Cookies: [1, 2, 3]
Sorted: [1, 2], [1, 2, 3]
i=0: 1>=1 → satisfied, i=1, j=1
i=1: 2>=2 → satisfied, i=2, j=2
Result: 2
```

Solution Approach

Concept: Sort both, assign smallest cookie that satisfies greed.

Algorithm

```
Algorithm ASSIGN_COOKIES(greed, cookies):
          Sort(greed), Sort(cookies)
          i = 0, j = 0

          While i < greed.length and j < cookies.length:
              If cookies[j] >= greed[i]:
                  i++
              j++

          Return i
```

Java Implementation

```
class Solution {
          public int findContentChildren(int[] g, int[] s) {
              Arrays.sort(g);
              Arrays.sort(s);

                int i = 0, j = 0;
                while (i < g.length && j < s.length) {
                    if (s[j] >= g[i]) {
                         i++;
                    }
                    j++;
                }
                return i;
          }
      }
```

Time & Space Complexity

Time Complexity: O(n log n + m log m) | Space Complexity: O(1)

229. Fractional Knapsack Problem

Understanding the Problem NEW

You have a knapsack with a weight capacity, and a set of items each with a weight and value — unlike the classic 0/1 Knapsack, here you're allowed to take a *fraction* of an item (like pouring out part of a sack of grain). Maximize the total value you can carry.

Brute Force Approach NEW

Try every possible combination of fractions for each item and compute total value — since fractions are continuous, there are infinitely many combinations to consider without a clear strategy, making truly brute enumeration impossible; even a coarse discretized brute-force would be extremely expensive.

Why We Optimize NEW

Because you can take *any fraction* of an item, the right measure of an item's 'efficiency' is its **value-per-unit-weight ratio** (value/weight). Sort items by this ratio in descending order, then greedily take as much as possible of the highest-ratio item first, then the next, and so on, taking only a fraction of the last item that fits when the capacity runs out. Because fractional taking is allowed, there's no risk of 'almost fitting but not quite' the way there is in 0/1 Knapsack — greedily maximizing value-density at every step is always optimal here, achieving O(n log n) for the sort, whereas the 0/1 version (no fractions allowed) genuinely requires the more expensive Dynamic Programming approach covered in the next chapter, since greedy can fail there.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Maximize value in knapsack by taking fractions of items.

Visual Diagram

```
Items: (value, weight)
       Item1: (60, 10) → value/weight = 6
       Item2: (100, 20) → value/weight = 5
       Item3: (120, 30) → value/weight = 4
       Knapsack Capacity: 50
```

```
       Sort by value/weight descending:
       Item1: 6, Item2: 5, Item3: 4

       Take Item1: weight=10, value=60
       Take Item2: weight=20, value=100 (total weight=30, value=160)
       Take 20/30 of Item3: weight=20, value=80 (total weight=50)
       Total value = 240

       Fractional knapsack can take partial items.
```

Solution Approach

Concept: Sort by value/weight ratio.

Algorithm

```
Algorithm FRACTIONAL_KNAPSACK(items, capacity):
           Sort items by value/weight ratio descending
           totalValue = 0

           For each item:
               If capacity >= item.weight:
                   totalValue += item.value
                   capacity -= item.weight
               Else:
                   totalValue += item.value * (capacity / item.weight)
                   break

           Return totalValue
```

Java Implementation

```
class Solution {
           public double fractionalKnapsack(int[] values, int[] weights, int
capacity) {
               int n = values.length;
               Item[] items = new Item[n];
               for (int i = 0; i < n; i++) {
                   items[i] = new Item(values[i], weights[i]);
               }

                 Arrays.sort(items, (a, b) -> Double.compare(b.ratio, a.ratio));

                 double totalValue = 0;
                 int remainingCapacity = capacity;

                 for (Item item : items) {
                     if (remainingCapacity >= item.weight) {
                         totalValue += item.value;
```

```
                        remainingCapacity -= item.weight;

                } else {
                    totalValue += item.value * ((double) remainingCapacity /
item.weight);
                    break;
                }
            }
            return totalValue;
        }

        class Item {
            int value, weight;
            double ratio;
            Item(int value, int weight) {
                this.value = value;
                this.weight = weight;
                this.ratio = (double) value / weight;
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(n log n) | Space Complexity: O(n)

230. Minimum Number of Coins

Understanding the Problem NEW

Given coin denominations and a target amount, find the minimum number of coins needed to make that amount (assuming unlimited coins of each denomination).

Brute Force Approach NEW

Try every possible combination of coins that sums to the target and find the one using the fewest coins — this is the general Coin Change problem, and exhaustively trying every combination is exponential.

Why We Optimize NEW

For 'nice' denomination systems like standard currency (1, 5, 10, 25...), a greedy approach works: always take as many of the **largest** denomination as possible before moving to smaller ones. This greedy approach is fast (O(number of denominations)) and gives the optimal answer for standard currency systems — but it's important to know this greedy approach is *not always correct* for arbitrary denominations (e.g. coins {1, 3, 4} making 6: greedy gives 4+1+1=3 coins, but the optimal is 3+3=2 coins) — this is a valuable cautionary example showing greedy isn't universally safe, and the general Coin Change problem actually needs Dynamic Programming (covered in the next chapter) to be solved correctly for arbitrary denominations.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find minimum number of coins to make amount.

Visual Diagram

```
Currency: [1, 2, 5, 10, 20, 50, 100, 500, 1000]
       Amount: 93

       Greedy:
       100 > 93 → skip
       50 ≤ 93 → take 50, remaining=43
       20 ≤ 43 → take 20, remaining=23
```

```
20 ≤ 23 → take 20, remaining=3
2 ≤ 3 → take 2, remaining=1
1 ≤ 1 → take 1, remaining=0

Coins: [50, 20, 20, 2, 1] → 5 coins

Amount: 49
40? No 40 coin doesn't exist
50 > 49 → skip
20 ≤ 49 → take 20, remaining=29
20 ≤ 29 → take 20, remaining=9
5 ≤ 9 → take 5, remaining=4
2 ≤ 4 → take 2, remaining=2
2 ≤ 2 → take 2, remaining=0
Coins: [20, 20, 5, 2, 2] → 5 coins
```

Solution Approach

Concept: Use largest denominations first.

Algorithm

```
Algorithm MIN_COINS(amount, denominations):
           coins = []
           For denom in denominations descending:
               While amount >= denom:
                   coins.add(denom)
                   amount -= denom
           Return coins
```

Java Implementation

```
class Solution {
           public int[] minCoins(int amount) {
               int[] denominations = {1000, 500, 100, 50, 20, 10, 5, 2, 1};
               List<Integer> coins = new ArrayList<>();

                 for (int denom : denominations) {
                     while (amount >= denom) {
                         coins.add(denom);
                         amount -= denom;
                     }
                 }

                 return coins.stream().mapToInt(i -> i).toArray();
           }
      }
```

Time & Space Complexity

Time Complexity: O(amount/min_coin) | Space Complexity: O(number of coins)

231. Lemonade Change

Understanding the Problem NEW

Customers pay for a $5 lemonade with a $5, $10, or $20 bill, one at a time in a given order — determine if you can always provide correct change (starting with no bills in hand) for every customer.

Brute Force Approach NEW

There's no meaningfully different 'brute force' here since the transactions arrive in a fixed order and must be handled as they come — the real question is what change-giving strategy to use.

Why We Optimize NEW

A greedy rule works: track how many $5 and $10 bills you're currently holding. When someone pays with $10, give back a $5 (only option). When someone pays with $20, prefer giving back one $10 + one $5 **if you have both**, rather than three $5s — because $10 bills are 'less flexible' (they can only be used as change for a $20, whereas $5 bills can be used as change for both $10 and $20) — saving your more flexible $5 bills for future customers whenever there's a choice. This greedy 'preserve flexibility' choice, verified in a single O(n) pass, correctly determines feasibility without needing to explore alternate change-giving choices.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Can we give change with only $5, $10, $20 bills?

Visual Diagram

```
Bills: [5, 5, 5, 10, 20]
       Output: true

       Step-by-step:
       5 → $5: count5=1
       5 → $5: count5=2
       5 → $5: count5=3
       10 → give $5 change: count5=2, count10=1
```

```
20 → give $15 change (10+5): count5=1, count10=0
Result: true

Bills: [5, 5, 10, 10, 20]
20 → need $15 change, have 10+5 → count5=1, count10=1
Wait, after [5,5,10,10]: count5=2, count10=2
20 → give 10+5 → count5=1, count10=1
Result: true

Bills: [5, 5, 10, 20]
20 → need $15, have 10+5 → give → true

Bills: [5, 5, 20]
20 → need $15, have no 10, only two 5s → 5+5=10 < 15 → false
```

Solution Approach

Concept: Track $5 and $10 bills, always use $10 first.

Algorithm

```
Algorithm LEMONADE_CHANGE(bills):
           count5 = 0, count10 = 0

           For bill in bills:
               If bill == 5:
                   count5++
               Else If bill == 10:
                   If count5 == 0: return false
                   count5--
                   count10++
               Else: // bill == 20
                   If count10 > 0 and count5 > 0:
                       count10--
                       count5--
                   Else If count5 >= 3:
                       count5 -= 3
                   Else:
                       return false

           Return true
```

Java Implementation

```
class Solution {
           public boolean lemonadeChange(int[] bills) {
               int count5 = 0, count10 = 0;

              for (int bill : bills) {
                  if (bill == 5) {
```

```
                count5++;
            } else if (bill == 10) {
                if (count5 == 0) return false;
                count5--;
                count10++;
            } else { // bill == 20
                if (count10 > 0 && count5 > 0) {
                    count10--;
                    count5--;
                } else if (count5 >= 3) {
                    count5 -= 3;
                } else {
                    return false;
                }
            }
        }
        return true;
    }
}
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

232. Valid Parenthesis Checker

Understanding the Problem NEW

Given a string containing '(', ')', and '*' (where '*' can act as either '(', ')', or an empty string), determine if the string can be interpreted as a valid parenthesis sequence.

Brute Force Approach NEW

Try every possible interpretation of each '*' character (as '(', ')', or empty) and check if any resulting string is validly balanced — with potentially many '*' characters, this is 3^(number of stars) combinations to check, exponential.

Why We Optimize NEW

Instead of resolving each '*' to one specific interpretation, track a **range** of possible 'open parenthesis count' values as you scan: a running minimum possible open-count (treating '*' as ')' or empty when it helps) and a running maximum possible open-count (treating '*' as '(' when it helps) — if this range ever excludes 0 from below (max drops below 0), the string is invalid; at the end, the string is valid if 0 falls within the final range. This tracks a whole range of possibilities simultaneously rather than exploring each concrete interpretation individually, collapsing an exponential search into a single O(n) pass.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Check if string with wildcard '*' can form valid parentheses.

Visual Diagram

```
Input: "()"
        Output: true

        Input: "(*)"
        Output: true ( '*' → ')' or empty)

        Input: "(*))"
        Output: true

        Greedy approach:
```

```
        Track min and max open parentheses
        minOpen = 0, maxOpen = 0

        For each char:
        If '(' : minOpen++, maxOpen++
        If ')' : minOpen--, maxOpen--
        If '*' : minOpen-- (treat as ')'), maxOpen++ (treat as '(')

        If maxOpen < 0: return false
        If minOpen < 0: minOpen = 0

        At end: minOpen == 0

        Example: "(*))"
        '(' : min=1, max=1
        '*' : min=0, max=2
        ')' : min=-1→0, max=1
        ')' : min=-1→0, max=0
        End: min=0 → true
```

Solution Approach

Concept: Track min and max open parentheses.

Algorithm

```
Algorithm VALID_PARENTHESIS(s):
            minOpen = 0, maxOpen = 0

            For char in s:
                If char == '(':
                    minOpen++
                    maxOpen++
                Else If char == ')':
                    minOpen--
                    maxOpen--
                Else: // '*'
                    minOpen-- // treat as ')'
                    maxOpen++ // treat as '('

                If maxOpen < 0: return false
                If minOpen < 0: minOpen = 0
            Return minOpen == 0
```

Java Implementation

```
class Solution {
          public boolean checkValidString(String s) {
              int minOpen = 0, maxOpen = 0;
```

```
            for (char c : s.toCharArray()) {
                if (c == '(') {
                    minOpen++;
                    maxOpen++;
                } else if (c == ')') {
                    minOpen--;
                    maxOpen--;
                } else { // '*'
                    minOpen--; // treat as ')'
                    maxOpen++; // treat as '('
                }

                if (maxOpen < 0) return false;
                if (minOpen < 0) minOpen = 0;
            }
            return minOpen == 0;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

233. N Meetings in One Room

Understanding the Problem NEW

Given start and end times for several meetings, find the maximum number of non-overlapping meetings that can be held in a single room.

Brute Force Approach NEW

Try every possible subset of meetings and check if they're pairwise non-overlapping, keeping the largest valid subset found — exponential in the number of meetings.

Why We Optimize NEW

Sort meetings by their **end time**, then greedily select each meeting in that order as long as it starts after the previously selected meeting ends. Choosing the meeting that *ends earliest* at every step (rather than, say, the shortest meeting, or the one starting earliest) leaves the most remaining room in the day for future meetings — this classic greedy strategy (a specific instance of 'activity selection') is provably optimal, turning an exponential subset search into an O(n log n) sort-then-scan.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Maximum number of meetings that can be scheduled.

Visual Diagram

```
Meetings: (start, end)
       (1, 2), (3, 4), (0, 6), (5, 7), (8, 9), (5, 9)
       Output: 4 (meetings 1,2,4,5)

       Sort by end time:
       (1,2), (3,4), (0,6), (5,7), (8,9), (5,9)

       Select:
       (1,2) → end=2
       (3,4) → end=4
       (5,7) → end=7
       (8,9) → end=9
```

```
       Total = 4

       Meeting (0,6) conflicts with (3,4) and (5,7), skipped
       Meeting (5,9) conflicts with (8,9), skipped
```

Solution Approach

Concept: Sort by end time, select non-conflicting.

Algorithm

```
Algorithm MAX_MEETINGS(meetings):
           Sort meetings by end time
           count = 1, lastEnd = meetings[0].end

           For i = 1 to n-1:
               If meetings[i].start > lastEnd:
                   count++
                   lastEnd = meetings[i].end

           Return count
```

Java Implementation

```
class Solution {
           public int maxMeetings(int[][] meetings) {
               Arrays.sort(meetings, (a, b) -> a[1] - b[1]);

                 int count = 1;
                 int lastEnd = meetings[0][1];

                 for (int i = 1; i < meetings.length; i++) {
                     if (meetings[i][0] > lastEnd) {
                         count++;
                         lastEnd = meetings[i][1];
                     }
                 }
                 return count;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n log n) | Space Complexity: O(1)

234. Jump Game

Understanding the Problem NEW

Given an array where each element represents the maximum jump length from that position, determine if you can reach the last index starting from the first.

Brute Force Approach NEW

Try every possible sequence of jump choices from the start, exploring all reachable positions recursively — exponential, since at each position there can be multiple possible jump lengths to try.

Why We Optimize NEW

You don't need to track *every* possible path — you only need to track the single number **'furthest index reachable so far.'** Scan left to right, and at each position (as long as it's within the currently-known reachable range), update the furthest reachable index using that position's jump length; if you ever reach a position beyond the current furthest-reachable index before updating it, you're stuck and the answer is false. This reduces an exponential path-exploration problem to a single O(n) pass, tracking only the one running maximum that actually matters, rather than every individual path.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Check if you can reach the last index.

Visual Diagram

```
Array: [2, 3, 1, 1, 4]
       Output: true

       Step-by-step:
       i=0: reachable = 0+2 = 2
       i=1: reachable = max(2, 1+3) = 4
       i=2: reachable = max(4, 2+1) = 4
       i=3: reachable = max(4, 3+1) = 4
       i=4: reachable >= 4 → true

       Array: [3, 2, 1, 0, 4]
```

```
Output: false

i=0: reachable = 3
i=1: reachable = max(3, 1+2) = 3
i=2: reachable = max(3, 2+1) = 3
i=3: reachable = max(3, 3+0) = 3
i=4: 4 > reachable → false
```

Solution Approach

Concept: Track maximum reachable index.

Algorithm

```
Algorithm JUMP_GAME(nums):
           maxReach = 0
           For i = 0 to n-1:
               If i > maxReach: return false
               maxReach = Math.max(maxReach, i + nums[i])
           Return true
```

Java Implementation

```
class Solution {
           public boolean canJump(int[] nums) {
               int maxReach = 0;
               for (int i = 0; i < nums.length; i++) {
                   if (i > maxReach) return false;
                   maxReach = Math.max(maxReach, i + nums[i]);
               }
               return true;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

235. Jump Game 2

Understanding the Problem NEW

Same setup as Jump Game, but now find the **minimum number of jumps** needed to reach the last index (assuming it's always reachable).

Brute Force Approach NEW

Explore every possible sequence of jumps (as in problem 234) and track the shortest sequence length found — exponential.

Why We Optimize NEW

Think of this like a 'level by level' expansion (similar in spirit to Breadth-First Search, previewed here before the Graphs chapter): at each 'jump count' level, you know the range of indices reachable using exactly that many jumps; greedily extend the **furthest reachable boundary** using every position within the current level's range, and increment the jump count each time you need to move to the next level's boundary. This greedy boundary-expansion (rather than trying every jump-length choice explicitly) computes the minimum jumps in a single O(n) pass — foreshadowing how BFS naturally finds shortest paths level by level in the Graphs chapter ahead.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Minimum jumps to reach last index.

Visual Diagram

```
Array: [2, 3, 1, 1, 4]
       Output: 2 (2→1→4)

       Step-by-step:
       i=0: jumps=0, currEnd=0, farthest=0
       i=0: farthest=2, i==currEnd → jumps=1, currEnd=2
       i=1: farthest=4
       i=2: farthest=4, i==currEnd → jumps=2, currEnd=4
       i=3: farthest=4
       i=4: i==currEnd → jumps=2? Actually we reach end at 4
```

```
Algorithm:
jumps=0, currEnd=0, farthest=0
For i=0 to n-2:
    farthest = max(farthest, i + nums[i])
    If i == currEnd:
        jumps++
        currEnd = farthest
Return jumps

Array: [2, 3, 1, 1, 4]
i=0: farthest=2, i==currEnd(0) → jumps=1, currEnd=2
i=1: farthest=4
i=2: farthest=4, i==currEnd(2) → jumps=2, currEnd=4
i=3: farthest=4
End: return jumps=2
```

Solution Approach

Concept: BFS level approach with greedy jumps.

Algorithm

```
Algorithm JUMP_GAME_2(nums):
           jumps = 0, currEnd = 0, farthest = 0

           For i = 0 to n-2:
               farthest = Math.max(farthest, i + nums[i])
               If i == currEnd:
                   jumps++
                   currEnd = farthest

           Return jumps
```

Java Implementation

```
class Solution {
           public int jump(int[] nums) {
               int jumps = 0;
               int currEnd = 0;
               int farthest = 0;

                 for (int i = 0; i < nums.length - 1; i++) {
                     farthest = Math.max(farthest, i + nums[i]);
                     if (i == currEnd) {

                      jumps++;
                      currEnd = farthest;
                  }
              }
              return jumps;
```

```
            }
        }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

236. Minimum Number of Platforms Required

Understanding the Problem NEW

Given arrival and departure times for several trains, find the minimum number of platforms needed so that no two trains needing a platform at the same time are forced to share one.

Brute Force Approach NEW

For every point in time, count how many trains are 'in the station' simultaneously by checking every train's arrival/departure against that timestamp — very inefficient if times span a large range, or $O(n^2)$ if comparing every pair of trains.

Why We Optimize NEW

Sort arrival times and departure times **separately**, then use two pointers walking through both sorted lists simultaneously: whenever the next event is an arrival, increment a 'platforms currently needed' counter (and track the maximum ever reached); whenever the next event is a departure, decrement it. This merges the arrival and departure timelines using the same 'sorted, two-pointer merge' idea seen in earlier chapters (like the Union of Sorted Arrays, Arrays-chapter problem 16), achieving O(n log n) for the sort plus a single O(n) merge pass, instead of any pairwise or per-timestamp comparison.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Minimum platforms needed for all trains.

Visual Diagram

```
Arrival: [900, 940, 950, 1100, 1500, 1800]
       Departure: [910, 1200, 1120, 1130, 1900, 2000]
       Output: 3

       Sort arrival and departure separately:
       Arrivals: 900, 940, 950, 1100, 1500, 1800
       Departures: 910, 1120, 1130, 1200, 1900, 2000

       i=0, j=0: 900 <= 910 → platform=2, i=1
       i=1: 940 <= 910? No → platform=1, j=1
```

```
        i=1: 940 <= 1120 → platform=2, i=2
        i=2: 950 <= 1120 → platform=3, i=3
        i=3: 1100 <= 1120 → platform=4, i=4
        i=4: 1500 <= 1120? No → platform=3, j=2
        i=4: 1500 <= 1130? No → platform=2, j=3
        i=4: 1500 <= 1200? No → platform=1, j=4
        i=4: 1500 <= 1900 → platform=2, i=5
        End: max=4? Wait, let's re-calc

        Actually max platforms = 3
        Let's re-trace:
        900 (platform=1)
        940 (platform=2) - 910 departs
        950 (platform=3) - trains: 900,940,950
        1100 (platform=4?) - 940 departs at 1120, wait 940 departs at 1200
        Actually arrival: 900,940,950,1100
        Departures: 910,1200,1120,1130
        900 (p1), 940 (p2), 950 (p3), 1100 (p4) → 4 platforms
        But expected is 3? Let's check:
        After 900 arrives, 910 departs
        At 940: platform1 free (900 left at 910)
        So: 940 (p1), 950 (p2), 1100 (p3) → 3 platforms
```

Solution Approach

Concept: Two-pointer sorted arrays.

Algorithm

```
Algorithm MIN_PLATFORMS(arrival, departure):
           Sort(arrival), Sort(departure)
           i=0, j=0, platforms=0, maxPlatforms=0

           While i < n:
               If arrival[i] <= departure[j]:
                   platforms++
                   i++
                   maxPlatforms = max(maxPlatforms, platforms)
               Else:
                   platforms--
                   j++

           Return maxPlatforms
```

Java Implementation

```
class Solution {
           public int minPlatforms(int[] arrival, int[] departure) {
               Arrays.sort(arrival);
               Arrays.sort(departure);
```

```
            int i = 0, j = 0;
            int platforms = 0, maxPlatforms = 0;
            int n = arrival.length;

            while (i < n) {
                if (arrival[i] <= departure[j]) {
                    platforms++;
                    i++;
                    maxPlatforms = Math.max(maxPlatforms, platforms);
                } else {
                    platforms--;
                    j++;
                }
            }
            return maxPlatforms;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n log n) | Space Complexity: O(1)

237. Job Sequencing Problem

Understanding the Problem NEW

Given a set of jobs, each with a deadline and a profit, and assuming each job takes exactly one unit of time and only one job can run per unit of time, schedule jobs to maximize total profit (a job only counts if completed by its deadline).

Brute Force Approach NEW

Try every possible subset and ordering of jobs that respects deadlines, tracking maximum total profit — exponential, since both which jobs to include and how to order them matter.

Why We Optimize NEW

Sort jobs by **profit in descending order**, then greedily try to schedule each job (starting with the most profitable) into the **latest available time slot** at or before its deadline (using a simple array or a Union-Find structure to efficiently find the latest free slot). Prioritizing the highest-profit jobs first, and slotting each as late as possible (to leave earlier slots open for other jobs with tighter deadlines), is provably optimal — this reduces an exponential search over job subsets and orderings into an O(n log n) sort plus O(n) (or O(n α(n)) with Union-Find) slot assignment.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Maximize profit by scheduling jobs with deadlines.

Visual Diagram

```
Jobs: (id, deadline, profit)
       J1: (1, 4, 20), J2: (2, 1, 10), J3: (3, 1, 40)
       J4: (4, 1, 30), J5: (5, 4, 25)

       Sort by profit:
       J3: (3,1,40), J5: (5,4,25), J1: (1,4,20), J4: (4,1,30), J2: (2,1,10)

       Schedule:
       J3: deadline=1 → slot 1
       J5: deadline=4 → slot 4
```

```
       J4: deadline=1 → slot 1 full, try slot 0? Actually slots are 1-indexed
          deadline=1 → slot 1 full → skip
       J1: deadline=4 → slot 3 (slot 4 taken)
       J2: deadline=1 → skip

       Selected: J3 (40), J5 (25), J1 (20) → total=85

       Slots: [_, _, _, _]
       J3: deadline=1 → [J3, _, _, _]
       J5: deadline=4 → [J3, _, _, J5]
       J1: deadline=4 → [J3, _, J1, J5] (slot 3)
```

Solution Approach

Concept: Sort by profit, assign to latest available slot.

Algorithm

```
Algorithm JOB_SEQUENCING(jobs):
           Sort jobs by profit descending
           maxDeadline = max deadline
           slots = new int[maxDeadline+1]
           Arrays.fill(slots, -1)
           totalProfit = 0, count = 0

            For job in jobs:
                // Find available slot
                For i = job.deadline down to 1:
                    If slots[i] == -1:
                        slots[i] = job.id
                        totalProfit += job.profit
                        count++
                        break

            Return {count, totalProfit}
```

Java Implementation

```
class Solution {
           public int[] jobScheduling(int[][] jobs) {
               // jobs[i] = [id, deadline, profit]
               Arrays.sort(jobs, (a, b) -> b[2] - a[2]);

                 int maxDeadline = 0;
                 for (int[] job : jobs) {

                  maxDeadline = Math.max(maxDeadline, job[1]);
              }

              int[] slots = new int[maxDeadline + 1];
```

```
            Arrays.fill(slots, -1);

            int count = 0, totalProfit = 0;
            for (int[] job : jobs) {
                for (int i = job[1]; i >= 1; i--) {
                    if (slots[i] == -1) {
                        slots[i] = job[0];
                        count++;
                        totalProfit += job[2];
                        break;
                    }
                }
            }
            return new int[]{count, totalProfit};
        }
    }
```

Time & Space Complexity

Time Complexity: O(n * maxDeadline) | Space Complexity: O(maxDeadline)

238. Candy

Understanding the Problem NEW

Children are lined up, each with a rating; distribute candies such that every child gets at least one, and any child with a strictly higher rating than an immediate neighbour gets more candies than that neighbour. Minimize total candies distributed.

Brute Force Approach NEW

Try every possible candy distribution satisfying the neighbour constraints and find the one with minimum total — an enormous search space since candy counts aren't bounded in advance.

Why We Optimize NEW

Two greedy passes handle both directions of the constraint independently: a **left-to-right pass** ensures every child with a higher rating than their left neighbour gets more candy than them (only checking the left-side constraint); a **right-to-left pass** similarly ensures the right-side constraint, taking the *maximum* of what each pass independently determined for every child (since both constraints must be satisfied simultaneously). Splitting a two-directional constraint into two independent one-directional passes, then combining results with a max, avoids ever needing to explore alternative distributions — achieving the minimum-total answer in O(n), just two linear scans.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Minimum candies to distribute to children with ratings.

Visual Diagram

```
Ratings: [1, 0, 2]
       Output: 5

       Candies: [2, 1, 2] → 2+1+2=5

       Step 1: Initialize all to 1: [1,1,1]
       Step 2: Left to right (if higher rating, more candy):
```

```
        i=1: 0 < 1 → no change
        i=2: 2 > 0 → candy[2] = candy[1]+1 = 2
        Result: [1,1,2]

        Step 3: Right to left (if higher rating):
        i=1: 0 < 1 → no change
        i=0: 1 > 0 → candy[0] = max(1, candy[1]+1) = max(1,2)=2
        Result: [2,1,2]
        Total: 5

        Ratings: [1, 2, 2]
        Output: 4

        Step 1: [1,1,1]
        Step 2 left-right: [1,2,1]
        Step 3 right-left: [1,2,1] (2=2 no change)
        Total: 4
```

Solution Approach

Concept: Two passes (left-to-right, right-to-left).

Algorithm

```
Algorithm CANDY(ratings):
           n = ratings.length
           candies = new int[n]
           Arrays.fill(candies, 1)

            // Left to right
            For i = 1 to n-1:
                If ratings[i] > ratings[i-1]:
                    candies[i] = candies[i-1] + 1

            // Right to left
            For i = n-2 down to 0:
                If ratings[i] > ratings[i+1]:
                    candies[i] = Math.max(candies[i], candies[i+1] + 1)

            Return sum(candies)
```

Java Implementation

```
class Solution {
           public int candy(int[] ratings) {
               int n = ratings.length;
               int[] candies = new int[n];
               Arrays.fill(candies, 1);

              // Left to right
```

```
            for (int i = 1; i < n; i++) {
                if (ratings[i] > ratings[i - 1]) {
                    candies[i] = candies[i - 1] + 1;
                }
            }

            // Right to left
            for (int i = n - 2; i >= 0; i--) {
                if (ratings[i] > ratings[i + 1]) {
                    candies[i] = Math.max(candies[i], candies[i + 1] + 1);
                }
            }

            int total = 0;
            for (int candy : candies) {
                total += candy;
            }
            return total;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

239. Shortest Job First (SJF) Scheduling

Understanding the Problem NEW

Given a set of jobs with burst times (how long each takes to run) arriving at the same time, schedule them one at a time (non-preemptively) to minimize average waiting time across all jobs.

Brute Force Approach NEW

Try every possible ordering of jobs and compute average waiting time for each — O(n!) orderings, since waiting time depends on the full sequence.

Why We Optimize NEW

The provably optimal greedy rule: always run the job with the **shortest burst time** next among those currently available. Intuitively, a job with a short burst time delays every job behind it by only a little, whereas a long job run early delays every subsequent job significantly — so clearing short jobs first minimizes the cumulative waiting incurred by everyone still waiting. Sorting jobs by burst time and running them in that order achieves optimal average waiting time in O(n log n), instead of exploring O(n!) possible schedules.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Calculate average waiting time for SJF scheduling.

Visual Diagram

```
Processes: [4, 3, 7, 1, 2] (burst times)
       SJF Order: [1, 2, 3, 4, 7]

       Completion times:
       1: 1
       2: 1+2=3
       3: 1+2+3=6
       4: 1+2+3+4=10
       7: 1+2+3+4+7=17

       Turnaround times:
       1: 1, 2: 3, 3: 6, 4: 10, 7: 17
```

```
       Waiting times = Turnaround - Burst:
       1:0, 2:1, 3:3, 4:6, 7:10
       Average waiting = (0+1+3+6+10)/5 = 4

       Preemptive SJF (Shortest remaining time first)
       Non-preemptive SJF (As above)
```

Solution Approach

Concept: Sort by burst time.

Algorithm

```
Algorithm SJF(burstTimes):
           Sort(burstTimes)
           waitingTime = 0, totalWaiting = 0

           For i = 0 to n-1:
               totalWaiting += waitingTime
               waitingTime += burstTimes[i]

           Return totalWaiting / n
```

Java Implementation

```
class Solution {
           public double sjf(int[] burstTimes) {
               Arrays.sort(burstTimes);
               double totalWaiting = 0;
               double waitingTime = 0;

                 for (int burst : burstTimes) {
                     totalWaiting += waitingTime;
                     waitingTime += burst;
                 }
                 return totalWaiting / burstTimes.length;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n log n) | Space Complexity: O(1)

241. Insert Interval

Understanding the Problem NEW

Given a list of non-overlapping intervals already sorted by start time, insert a new interval into the correct position, merging with any existing intervals it now overlaps.

Brute Force Approach NEW

Add the new interval to the list, then re-sort everything and re-run a full merge-overlapping-intervals pass from scratch — O(n log n), redoing sorting work that was already done, since the input was already sorted before the insertion.

Why We Optimize NEW

Because the existing intervals are already sorted and non-overlapping, you can do this in a single O(n) pass without re-sorting: copy over every interval that ends entirely before the new interval starts (unaffected, no merge needed); then merge the new interval with every interval it overlaps (expanding its bounds as you absorb each one); then copy over every remaining interval that starts entirely after the merged result. This reuses the fact the input was already sorted to avoid the O(n log n) resort that treating this as 'insert then fully re-merge from scratch' would require.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Insert new interval into sorted non-overlapping intervals.

Visual Diagram

```
Intervals: [[1,3], [6,9]]
       New Interval: [2,5]
       Output: [[1,5], [6,9]]

       Merged: [1,3] overlaps with [2,5] → [1,5]
       [6,9] no overlap → keep

       Intervals: [[1,2], [3,5], [6,7], [8,10]]
       New: [4,8]
       Output: [[1,2], [3,10]]
```

```
[1,2] no overlap → add
[3,5] overlaps with [4,8] → merge → [3,8]
[6,7] overlaps with [3,8] → merge → [3,8]
[8,10] overlaps with [3,8] → merge → [3,10]
Result: [[1,2], [3,10]]
```

Solution Approach

Concept: Merge overlapping intervals after insertion.

Algorithm

```
Algorithm INSERT_INTERVAL(intervals, newInterval):
           result = []
           i = 0, n = intervals.length

           // Add all intervals before newInterval
           While i < n and intervals[i][1] < newInterval[0]:
               result.add(intervals[i])
               i++

           // Merge overlapping intervals
           While i < n and intervals[i][0] <= newInterval[1]:
               newInterval[0] = min(newInterval[0], intervals[i][0])
               newInterval[1] = max(newInterval[1], intervals[i][1])
               i++
           result.add(newInterval)

           // Add remaining intervals
           While i < n:
               result.add(intervals[i])
               i++

           Return result
```

Java Implementation

```
class Solution {
           public int[][] insert(int[][] intervals, int[] newInterval) {
               List<int[]> result = new ArrayList<>();
               int i = 0, n = intervals.length;

                 // Add all intervals before newInterval
                 while (i < n && intervals[i][1] < newInterval[0]) {

                  result.add(intervals[i++]);
              }

              // Merge overlapping intervals
```

```
            while (i < n && intervals[i][0] <= newInterval[1]) {
                newInterval[0] = Math.min(newInterval[0], intervals[i][0]);
                newInterval[1] = Math.max(newInterval[1], intervals[i][1]);
                i++;
            }
            result.add(newInterval);

            // Add remaining intervals
            while (i < n) {
                result.add(intervals[i++]);
            }

            return result.toArray(new int[result.size()][]);
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

242. Merge Intervals

Understanding the Problem NEW

Given a list of intervals (not necessarily sorted or non-overlapping), merge all overlapping ones into the minimal set of non-overlapping intervals. (This is the same problem as the Arrays chapter's problem 42, revisited here in the context of greedy interval techniques.)

Brute Force Approach NEW

Compare every pair of intervals to check for overlap and merge repeatedly — potentially $O(n^2)$ or worse, since merging one pair can create new overlaps needing further merging.

Why We Optimize NEW

Exactly the same technique as Arrays-chapter problem 42: **sort by start time** first, then a single pass only ever needs to compare each interval against the most recently merged one (extending it if overlapping, or starting a new merged interval otherwise) — O(n log n) for the sort plus O(n) for the merge pass, avoiding the need to compare every pair directly.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Merge all overlapping intervals.

Visual Diagram

```
Intervals: [[1,3], [2,6], [8,10], [15,18]]
       Output: [[1,6], [8,10], [15,18]]

       Sort by start:
       [[1,3], [2,6], [8,10], [15,18]]

       [1,3] → current
       [2,6] overlaps → [1,6]
       [8,10] no overlap → add [1,6], current=[8,10]
       [15,18] no overlap → add [8,10], current=[15,18]
       Add [15,18]
```

```
Intervals: [[1,4], [4,5]]
Output: [[1,5]]

[1,4] and [4,5] overlap at 4 → merge → [1,5]
```

Solution Approach

Concept: Sort by start, merge overlapping.

Algorithm

```
Algorithm MERGE_INTERVALS(intervals):
           Sort intervals by start
           result = []
           current = intervals[0]

           For i = 1 to n-1:
               If intervals[i][0] <= current[1]:
                   current[1] = max(current[1], intervals[i][1])
               Else:
                   result.add(current)
                   current = intervals[i]
           result.add(current)
           Return result
```

Java Implementation

```
class Solution {
           public int[][] merge(int[][] intervals) {
               Arrays.sort(intervals, (a, b) -> a[0] - b[0]);

                 List<int[]> result = new ArrayList<>();
                 int[] current = intervals[0];

                 for (int i = 1; i < intervals.length; i++) {
                     if (intervals[i][0] <= current[1]) {
                         current[1] = Math.max(current[1], intervals[i][1]);
                     } else {
                         result.add(current);
                         current = intervals[i];
                     }
                 }
                 result.add(current);

               return result.toArray(new int[result.size()][]);
           }
       }
```

Time & Space Complexity

Time Complexity: O(n log n) | Space Complexity: O(n)

243. Non-Overlapping Intervals

Understanding the Problem NEW

Given a set of intervals, find the minimum number of intervals to remove so that none of the remaining ones overlap.

Brute Force Approach NEW

Try every possible subset of intervals to remove and check if what remains is overlap-free, tracking the smallest removal count — exponential.

Why We Optimize NEW

This is really the 'N Meetings in One Room' problem (233) turned inside-out: instead of asking 'how many non-overlapping intervals can I keep,' ask 'how many can I keep' using the exact same greedy technique (sort by **end time**, greedily keep an interval if it doesn't conflict with the last kept one), and then the minimum removals is simply `total intervals - maximum keepable`. Recognizing this problem as the complement of one you've already solved, rather than solving it from scratch, again turns an exponential subset search into an O(n log n) sort-and-scan.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Minimum intervals to remove to make non-overlapping.

Visual Diagram

```
Intervals: [[1,2], [2,3], [3,4], [1,3]]
      Output: 1 (remove [1,3])

      Sort by end:
      [[1,2], [2,3], [1,3], [3,4]]
      [1,2] → end=2, count=1
      [2,3] → start(2) >= end(2) → end=3, count=2
      [1,3] → start(1) < end(3) → skip, removals=1
      [3,4] → start(3) >= end(3) → end=4, count=3
      Removals = n - count = 4-3=1

      Intervals: [[1,2], [1,2], [1,2]]
```

```
      Sort: [[1,2], [1,2], [1,2]]
      [1,2] → end=2
      [1,2] → skip → removals=1
      [1,2] → skip → removals=2
      Result: 2
```

Solution Approach

Concept: Sort by end time, keep maximum non-overlapping.

Algorithm

```
Algorithm NON_OVERLAPPING_INTERVALS(intervals):
          Sort intervals by end
          count = 1, end = intervals[0][1]

          For i = 1 to n-1:
              If intervals[i][0] >= end:
                  count++
                  end = intervals[i][1]

          Return n - count
```

Java Implementation

```
class Solution {
          public int eraseOverlapIntervals(int[][] intervals) {
              Arrays.sort(intervals, (a, b) -> a[1] - b[1]);

                 int count = 1;
                 int end = intervals[0][1];

                 for (int i = 1; i < intervals.length; i++) {
                     if (intervals[i][0] >= end) {
                         count++;
                         end = intervals[i][1];
                     }
                 }
                 return intervals.length - count;
          }
      }
```

Time & Space Complexity

Time Complexity: O(n log n) | Space Complexity: O(1)

Greedy Summary

Problem	Difficulty	Time	Space	Key Technique
Assign Cookies	Easy	O(n log n)	O(1)	Sort both, match
Fractional Knapsack	Easy	O(n log n)	O(n)	Sort by ratio
Min Coins	Easy	O(amount/min)	O(n)	Largest first
Lemonade Change	Easy	O(n)	O(1)	Track bills
Valid Parenthesis	Easy	O(n)	O(1)	Min/max open
N Meetings	Medium	O(n log n)	O(1)	Sort by end
Jump Game	Medium	O(n)	O(1)	Max reachable
Jump Game 2	Medium	O(n)	O(1)	BFS level
Min Platforms	Medium	O(n log n)	O(1)	Two pointers
Job Sequencing	Medium	O(n log n)	O(maxDeadline)	Sort profit + slots
Candy	Medium	O(n)	O(n)	Two passes
SJF Scheduling	Medium	O(n log n)	O(1)	Sort burst times
Insert Interval	Medium	O(n)	O(n)	Merge on fly
Merge Intervals	Medium	O(n log n)	O(n)	Sort + merge
Non-overlapping	Hard	O(n log n)	O(1)	Sort by end